

Logan MacAskill

logan.macaskill.com
Los Altos, CA

github.com/log2bits
linkedin.com/in/logan-macaskill-356190222

(650) 237-9593
logan@macaskill.com

About Me

Third-year CS student with two software engineering internships spent shipping and tuning production systems. I love problem solving, especially when I can take complete advantage of the hardware. I've worked across realtime systems, graphics, AI, and data science, and I'm currently building my own Vulkan & C++ deferred renderer.

Experience

Paid Software Engineering Intern **Flickr** **Jun 2023 - Aug 2023**

- Cut compute cost 92% and raised throughput 39% on Wolverine, a Python AWS Lambda photo-repair service running over a library of ~10 billion photos, by tuning memory and thread allocation to the cheapest stable configuration.
- Profiled dozens of Lambda configurations on controlled batches of real jobs in Splunk, modeling cost per job, throughput, and tail latency to find the cost/performance point AWS docs do not surface.
- Recovered AWS's undocumented vCPU-to-memory scaling ratio by measuring CPU behavior across memory settings and fitting a linear regression, enabling accurate per-job cost estimates.

Paid Software Engineering Intern **SmugMug** **Jun 2022 - Aug 2022**

- Owned a user-facing gallery stats feature end to end (PHP, React, Next.js), adding daily refresh scheduling and clearer labels and empty states, shipped through PR review and QA.
- Wrote unit tests and backend changes supporting a release-critical PHP 8.1 upgrade, pairing with senior engineers to land it on schedule.

Projects

ray-vox: Ray-traced Voxel Renderer **Rust, WebGPU, Data Structures, Optimization** **2026**

- Packed an 84 MB voxel scene into 16 MB, smaller than zstd at max (19 MB) and still directly GPU-traversable, with a custom sparse-voxel structure for a from-scratch ray-traced renderer in Rust and WebGPU.
- Eliminated all per-node child pointers using per-node bitmasks and bit-counting, and packed node offsets into a single 32-bit word, cutting memory while keeping ray traversal branch-light for GPU warps.
- Designed the format as a GPU acceleration structure with built-in level-of-detail and matching on-disk and in-GPU-memory layout, so uploads are near-zero-conversion copies.

Sanity Check: Logic Puzzle Game **Unity, C#, Constraint Solving, Procedural Generation** **2026**

- Built the puzzle engine for a five-person, one-week game jam entry: every possible answer is one bit in a hand-written bitset, each door's statement compiles to the set of answers it allows, and solving a room is set intersection.
- Generated puzzles that are provably unique *and* minimal, rejecting any room where a clue could be dropped without breaking it, and tuned five difficulty tiers on how many statements must combine before the first deduction is possible.
- Hand-wrote the set layer (64-bit words, SWAR popcount, short-circuiting overlap tests) for a generator that scores up to 20,000 candidate rooms per puzzle. Also owned the game's lighting, post-processing, and camera feel.

Crowd Surfers: Real-time 3D Game **Godot, Shaders, Lighting, Architecture** **2025 - 2026**

- Convinced a 100-person student team to rebuild the game's renderer mid-project, moving from a faked-depth sprite system to true 3D with an angled orthographic camera, by building the prototype instead of arguing for it.
- Built a 3D occlusion-based transparency shader that fades buildings as the player skates behind them, using a camera-to-player frustum test plus dithered alpha to fit the alpha-cut asset pipeline.

Real-time Dielectric Spectral Raymarcher **GLSL, Spectral Rendering, Sampling, GPU** **2026**

- Rendered a physically based dispersive diamond in one real-time GLSL shader: 65,536 distinct wavelengths resolved from two samples per pixel, no denoiser and no accumulation buffer, by scheduling wavelengths across space with a 16x16 Bayer dither.
- Replaced raymarching with exact ray-plane intersection across the icosahedron's 20 faces, and made light transport deterministic by peeling exact Fresnel energy at every facet instead of stochastically sampling one path: noise-free at one sample, with surface normals free.
- Held real-time frame rates in a browser tab by cutting each ray once trapped energy fell below 4%, and keeping every pixel a self-contained shader invocation with no frame history and no neighbor reads.

Coaxial Swerve Drive **Java, Control Theory, Computer Vision** **2023 - 2024**

- Led an off-season FRC team to a working coaxial swerve prototype in one month, about four months ahead of schedule, building the custom CNC chassis, electronics, and full software stack (~2,000 lines of Java) with no prior team experience.
- Held sub-centimeter localization through wheel slip by fusing wheel encoders, gyro, and AprilTag vision in a Kalman-filter pose estimator, with per-module PID and feedforward on a 1 kHz control loop after self-teaching graduate-level control theory.
- Simulated the full robot (torque curves, inertia, battery, brownouts) on the exact production code, and derived current limits from mass and tire friction to maximize grip without slip or brownout.

Education

University of California **Santa Cruz, CA** **Expected Jun 2028**

B.S. in Computer Science | GPA: 3.6/4.0

Honors: Merit Scholarship, Dean's Honors

Relevant Coursework: Data Structures & Algorithms, Computer Architecture, Systems Programming in C, Linear Algebra, Vector Calculus, Probability & Statistics

Activities: UC Santa Cruz Game Design & Art Club

Skills

Languages: Rust, C++, C#, Python, Java, TypeScript / JavaScript, PHP, GLSL / WGSL

Tools & Cloud: Git, Linux, AWS (EC2, Lambda, Graviton), Docker, OpenCV, Godot, Unity, React / Next.js

Concepts: Performance Optimization, GPU Compute, Control Theory, Computer Vision, Pose Estimation

Graphics & GPU: Vulkan, WebGPU, Ray Tracing, Real-Time Rendering, Rasterization, Shaders, Level-of-Detail